Lab Sheet 03

Title: NoSQL Database (MongoDB)

Objective:

- Understand about Insert and view.

Customer Orders Table

OrderID	CustomerName	Product	Quantity	Price (USD)	OrderDate	Status
101	Alice Brown	Laptop	1	1200.00	2024-03-01	Shipped
102	Bob Smith	Smartphone	2	699.99	2024-03-02	Pending
103	Charlie Lee	Headphones	3	49.99	2024-03-03	Delivered
104	David Wong	Monitor	1	250.00	2024-03-04	Shipped
105	Emma Johnson	Keyboard	2	79.99	2024-03-05	Canceled
106	Frank Adams	Mouse	1	29.99	2024-03-06	Pending
107	Grace White	Tablet	1	499.99	2024-03-07	Delivered
108	Henry Clark	Smartwatch	1	199.99	2024-03-08	Shipped
109	Ivy Turner	Laptop	1	1299.00	2024-03-09	Delivered
110	Jack Miller	Wireless Earbuds	2	99.99	2024-03-10	Pending

1. Create a Database

```
use EcomerceDB
```

2. Show All Databases

```
test> show dbs
CompanyDB      72.00 KiB
EcomerceDB     80.00 KiB
LibraryDB      72.00 KiB
OnlineStoreDB  72.00 KiB
SchoolDB       56.00 KiB
admin          40.00 KiB
config         48.00 KiB
local          80.00 KiB
test>
```

3. Create a Collection (Equivalent to Creating a Table in SQL)

```
db.createCollection("CustemerOrers")
show collections
```

4. Insert a Single Document (Order from Alice Brown)

```
db.CustomerOrders.insertOne({
  CustomerName: 'Alice Brown',
  Product: 'Laptop',
  Quantity: 1,
  Price: 1200.00,
  OrderDate: new Date('2024-03-01')
})
```

5. Insert Multiple Documents (Insert Many Orders at Once)

```
db.CustomerOrders.insertMany([
    {
        OrderID: 102,
        CustomerName: "Bob Smith",
        Product: "Smartphone",
        Quantity: 2,
        Price: 699.99,
        OrderDate: new Date("2024-03-02"),
        Status: "Pending"
    },
    {
        OrderID: 103,
        CustomerName: "Charlie Lee",
        Product: "Headphones",
        Quantity: 3,
        Price: 49.99,
        OrderDate: new Date("2024-03-03"),
        Status: "Delivered"
    },
    {
        OrderID: 104,
        CustomerName: "David Wong",
        Product: "Monitor",
        Quantity: 1,
```

```
        Price: 250.00,
        OrderDate: new Date("2024-03-04"),
        Status: "Shipped"
    },
    {
        OrderID: 105,
        CustomerName: "Emma Johnson",
        Product: "Keyboard",
        Quantity: 2,
        Price: 79.99,
        OrderDate: new Date("2024-03-05"),
        Status: "Canceled"
    },
    {
        OrderID: 106,
        CustomerName: "Frank Adams",
        Product: "Mouse",
        Quantity: 1,
        Price: 29.99,
        OrderDate: new Date("2024-03-06"),
        Status: "Pending"
    },
    {
        OrderID: 107,
        CustomerName: "Grace White",
        Product: "Tablet",
        Quantity: 1,
        Price: 499.99,
        OrderDate: new Date("2024-03-07"),
        Status: "Delivered"
    },
    {
        OrderID: 108,
        CustomerName: "Henry Clark",
        Product: "Smartwatch",
        Quantity: 1,
        Price: 199.99,
        OrderDate: new Date("2024-03-08"),
        Status: "Shipped"
    },
    {
        OrderID: 109,
        CustomerName: "Ivy Turner",
        Product: "Laptop",
        Quantity: 1,
```

```
        Price: 1299.00,
        OrderDate: new Date("2024-03-09"),
        Status: "Delivered"
    },
    {
        OrderID: 110,
        CustomerName: "Jack Miller",
        Product: "Wireless Earbuds",
        Quantity: 2,
        Price: 99.99,
        OrderDate: new Date("2024-03-10"),
        Status: "Pending"
    }
])
```

View Data

6. Find All Orders

```
db.CustomerOrders.find()
```

7. Find a Specific Order (Grace White's Order)

```
db.CustomerOrders.find({CustomerName:'Grace White'})
```

8. Find Orders Above a Certain Price ($500 and Above)

```
db.CustomerOrders.find({Price : {$gt:500}})
```

9. Find Orders with a Specific Status (Pending Orders)

```
db.CustomerOrders.find({Status : "Pending"})
```

10. Find Orders Placed Between Two Dates (March 3 to March 7, 2024)

```
db.CustomerOrders.find({
    OrderDate:{
        $gtr:new Date("2024-03-03"),
        $lte:new Date("2024-03-07")
    }
})
```

11. Find Orders in a Specific Price Range ($50 - $100)

```
db.CustomerOrders.find({
  Price: { $gte: 50, $lte: 100 }
})
```

```
db.CustomerOrders.find({
  Price: { $gte: 200, $lte: 800 }
})
```

12. Find Orders Using OR Condition (Shipped or Delivered Orders)

```
db.CustomerOrders.find({
  $or: [
    { Status: "Shipped" },
    { Status: "Delivered" }
  ]
})
```

13. Find Orders with Specific Products (Laptop or Smartphone)

```
db.CustomerOrders.find({
  $or: [
    { Product: "Laptop" },
    { Product: "Smartphone" }
  ]
})

db.CustomerOrders.find({
  Product: { $in: ["Laptop", "Smartphone"] }
})
```

14. Find Orders Where Quantity is Not 1

```
db.CustomerOrders.find({
  Quantity: { $ne: 1 }
})
```

Exercise 01

Employees Table

EmployeeID	Name	Department	Age	Salary (USD)	HireDate	Status
201	Alice Brown	IT	28	75000	2021-06-15	Active
202	Bob Smith	HR	35	65000	2019-09-22	Active
203	Charlie Lee	IT	40	85000	2018-04-10	Resigned
204	David Wong	Finance	45	95000	2016-12-05	Active
205	Emma Johnson	IT	29	72000	2020-02-18	Active
206	Frank Adams	Marketing	38	68000	2017-07-30	Resigned
207	Grace White	HR	31	70000	2019-10-10	Active
208	Henry Clark	IT	27	77000	2022-01-25	Active
209	Ivy Turner	Finance	33	80000	2021-03-12	Active
210	Jack Miller	Marketing	36	73000	2018-11-20	Active

1. Create a new database called **CompanyDB**.

```
use  CompanyDB
```

2. Create a collection named **Employees**.

```
db.createCollection("Employees")
```

3. Insert **all employee records** into the collection.

```
db.Employees.insertMany([
  { EmployeeID: 201, Name: "Alice Brown", Department: "IT", Age: 28, Salary: 75000,
HireDate: new Date("2021-06-15"), Status: "Active" },
  { EmployeeID: 202, Name: "Bob Smith", Department: "HR", Age: 35, Salary: 65000,
HireDate: new Date("2019-09-22"), Status: "Active" },
  { EmployeeID: 203, Name: "Charlie Lee", Department: "IT", Age: 40, Salary: 85000,
HireDate: new Date("2018-04-10"), Status: "Resigned" },
  { EmployeeID: 204, Name: "David Wong", Department: "Finance", Age: 45, Salary: 95000,
HireDate: new Date("2016-12-05"), Status: "Active" },
  { EmployeeID: 205, Name: "Emma Johnson", Department: "IT", Age: 29, Salary: 72000,
HireDate: new Date("2020-02-18"), Status: "Active" },
  { EmployeeID: 206, Name: "Frank Adams", Department: "Marketing", Age: 38, Salary: 68000,
HireDate: new Date("2017-07-30"), Status: "Resigned" },
  { EmployeeID: 207, Name: "Grace White", Department: "HR", Age: 31, Salary: 70000,
HireDate: new Date("2019-10-10"), Status: "Active" },
  { EmployeeID: 208, Name: "Henry Clark", Department: "IT", Age: 27, Salary: 77000,
HireDate: new Date("2022-01-25"), Status: "Active" },
```

```
  { EmployeeID: 209, Name: "Ivy Turner", Department: "Finance", Age: 33, Salary: 80000,
HireDate: new Date("2021-03-12"), Status: "Active" },
  { EmployeeID: 210, Name: "Jack Miller", Department: "Marketing", Age: 36, Salary: 73000,
HireDate: new Date("2018-11-20"), Status: "Active" }
])
```

4. Retrieve **all employees** in the collection.

```
db.Employees.find()
```

5. Find details of the employee **named "Alice Brown"**.

```
db.Employees.find({ Name: "Alice Brown" })
```

6. Find all employees working in the **IT department**.

```
db.Employees.find({ Department: "IT" })
```

7. Find employees who are **older than 30 years**.

```
db.Employees.find({ Age: { $gt: 30 } })
```

8. Find employees **earning more than $75,000**.

```
db.Employees.find({ Salary: { $gt: 75000 } })
```

9. Find employees who were **hired between 2019 and 2021**.

```
db.Employees.find({
  HireDate: {
    $gte: new Date("2019-01-01"),
    $lte: new Date("2021-12-31")
  }
})
```

10. Find employees whose **names start with "A"**.

```
db.Employees.find({
  Name: { $regex: "^A" }
})
```

11. Find employees who **do not belong to the IT department**.

```
db.Employees.find ({
  Department:{$ne:"IT"}
})
```

Exercise 02

Online Store Products Table

ProductID	ProductName	Category	Price (USD)	Stock	Rating	AddedDate
301	Laptop	Electronics	1200.00	15	4.5	2024-02-01
302	Smartphone	Electronics	699.99	25	4.7	2024-02-05
303	Headphones	Accessories	49.99	50	4.3	2024-02-10
304	Monitor	Electronics	250.00	10	4.6	2024-02-15
305	Keyboard	Accessories	79.99	30	4.2	2024-02-20
306	Mouse	Accessories	29.99	45	4.1	2024-02-25
307	Tablet	Electronics	499.99	20	4.4	2024-03-01
308	Smartwatch	Wearable Tech	199.99	18	4.5	2024-03-05
309	Wireless Earbuds	Accessories	99.99	35	4.3	2024-03-10
310	Gaming Chair	Furniture	250.00	12	4.6	2024-03-15

1. Create a new **database named "OnlineStoreDB"**.

```
use OnlineStoreDB
```

2. Create a **collection named "Products"**.

```
db.createCollection("Products")
```

3. Insert **a single product** (Laptop) into the collection.

```
db.Products.insertOne({
  ProductID: 301,
  ProductName: "Laptop",
  Category: "Electronics",
  Price: 1200.00,
  Stock: 15,
  Rating: 4.5,
  AddedDate: new Date("2024-02-01")
})
```

4. Insert **all products** into the collection at once.

```
db.Products.insertMany([
  { ProductID: 301, ProductName: "Laptop", Category: "Electronics", Price: 1200.00,
Stock: 15, Rating: 4.5, AddedDate: new Date("2024-02-01") },
```

```
  { ProductID: 302, ProductName: "Smartphone", Category: "Electronics", Price:
699.99, Stock: 25, Rating: 4.7, AddedDate: new Date("2024-02-05") },
  { ProductID: 303, ProductName: "Headphones", Category: "Accessories", Price: 49.99,
Stock: 50, Rating: 4.3, AddedDate: new Date("2024-02-10") },
  { ProductID: 304, ProductName: "Monitor", Category: "Electronics", Price: 250.00,
Stock: 10, Rating: 4.6, AddedDate: new Date("2024-02-15") },
  { ProductID: 305, ProductName: "Keyboard", Category: "Accessories", Price: 79.99,
Stock: 30, Rating: 4.2, AddedDate: new Date("2024-02-20") },
  { ProductID: 306, ProductName: "Mouse", Category: "Accessories", Price: 29.99,
Stock: 45, Rating: 4.1, AddedDate: new Date("2024-02-25") },
  { ProductID: 307, ProductName: "Tablet", Category: "Electronics", Price: 499.99,
Stock: 20, Rating: 4.4, AddedDate: new Date("2024-03-01") },
  { ProductID: 308, ProductName: "Smartwatch", Category: "Wearable Tech", Price:
199.99, Stock: 18, Rating: 4.5, AddedDate: new Date("2024-03-05") },
  { ProductID: 309, ProductName: "Wireless Earbuds", Category: "Accessories", Price:
99.99, Stock: 35, Rating: 4.3, AddedDate: new Date("2024-03-10") },
  { ProductID: 310, ProductName: "Gaming Chair", Category: "Furniture", Price:
250.00, Stock: 12, Rating: 4.6, AddedDate: new Date("2024-03-15") }
])
```

5. Retrieve **all products** from the collection.

```
db.Products.find()
```

6. Find the product **named "Tablet"**.

```
db.Products.find({ ProductName: "Tablet" })
```

7. Find all products in the **"Electronics" category**.

```
db.Products.find({ Category: "Electronics" })
```

8. Find products **priced above $500**.

```
db.Products.find({ Price: { $gt: 500 } })
```

9. Find products **with stock less than 20**.

```
db.Products.find({ Stock: { $lt: 20 } })
```

10. Find products that have a **rating of 4.5 or higher**.

```
db.Products.find({ Rating: { $gte: 4.5 } })
```

11. Find all **Accessories** and **Wearable Tech** products.

```
db.Products.find({
  Category: { $in: ["Accessories", "Wearable Tech"] }
})
```

12. Retrieve products **sorted by price in ascending order**.

```
db.Products.find().sort({ Price: 1 })
```

13. Find products **added after February 10, 2024**.

```
db.Products.find({
  AddedDate: { $gt: new Date("2024-02-10") }
})
```

14. Find products where **price is between $50 and $250**.

```
db.Products.find({
  Price: { $gte: 50, $lte: 250 }
})
```

15. Find products whose **name starts with "S"**.

```
db.Products.find({
  ProductName: { $regex: "^S" }
})
```

16. Find the **top 3 most expensive products**.

```
db.Products.find()
  .sort({ Price: -1 })
  .limit(3)
```